

1. Understanding Rate Limiting in Node.js / Express.js

1.1 What is Rate Limiting?

Rate limiting is a mechanism used to **control the number of requests a client can make** to an API within a specific time window. It helps protect your server from:

- Denial of Service (DoS) attacks
 - Brute-force attempts
 - Excessive API usage
 - Server overload
-

1.2 Common Rate Limiting Strategies

a) Fixed Window

Allow X requests per Y time window. Example: 100 requests per 10 minutes.

b) Sliding/ Rolling Window

Looks at the last time period relative to current user activity.

c) Token Bucket

Each request consumes tokens; tokens refill at fixed intervals.

d) Leaky Bucket

Requests enter a queue, processed at fixed rate.

2. Implementing Rate Limiting in Express.js

The most widely used package is:

```
express-rate-limit
```

Install:

```
npm install express-rate-limit
```

2.1 Basic Rate Limiting Setup

server.js

```
const express = require("express");
const rateLimit = require("express-rate-limit");

const app = express();

// Basic rate limiter: max 100 requests per 15 minutes
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // max requests per window
  message: "Too many requests, please try again later.",
  standardHeaders: true, // Return rate limit info in the RateLimit-*
  headers,
  legacyHeaders: false // Disable the X-RateLimit-* headers
});

// Apply to all requests
app.use(limiter);

app.get("/", (req, res) => {
  res.send("Welcome! You are under rate limit protection.");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

2.2 Creating Different Limits for Different Routes

```
// Rate limit for login route: strict
const loginLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  max: 5, // 5 requests per minute
  message: "Too many login attempts."
});

// Rate limit for public API: moderate
const apiLimiter = rateLimit({
  windowMs: 10 * 60 * 1000,
  max: 200
});

// Apply limiter only to /login route
app.post("/login", loginLimiter, (req, res) => {
  res.send("Login route");
});

app.use("/api/", apiLimiter);
```

2.3 Using Redis for Distributed Rate Limiting (Production)

When your app runs on multiple servers, in-memory rate limiting will break. Use Redis instead:

Install:

```
npm install rate-limit-redis redis
```

```
const RedisStore = require("rate-limit-redis");
const Redis = require("redis");

// Create Redis client
const redisClient = Redis.createClient({
  socket: { host: '127.0.0.1', port: 6379 }
});

redisClient.connect();

const limiter = rateLimit({
  store: new RedisStore({
    sendCommand: (...args) => redisClient.sendCommand(args)
  }),
  windowMs: 1 * 60 * 1000,
  max: 100
});

app.use(limiter);
```

3. Helmet.js — Security Middleware for Express.js

3.1 What is Helmet?

Helmet helps secure Express apps by setting appropriate **HTTP security headers**.

It protects against:

- Cross-Site Scripting (XSS)
- MIME sniffing
- Clickjacking
- Content Security Policy violations
- DNS prefetching
- Frame embedding
- Other common web vulnerabilities

Install:

```
npm install helmet
```

4. Implementing Helmet in Express.js

4.1 Basic Helmet Setup

```
const express = require("express");
const helmet = require("helmet");

const app = express();

app.use(helmet()); // Enable all default protections

app.get("/", (req, res) => {
  res.send("Helmet security enabled");
});

app.listen(3000, () => console.log("Server running"));
```

4.2 Helmet Modules Explained

Helmet provides several sub-modules:

Helmet Module	Purpose
helmet.contentSecurityPolicy()	Prevents XSS attacks
helmet.xssFilter()	Basic protection against reflected XSS
helmet.noSniff()	Prevent MIME type sniffing
helmet.frameguard()	Prevent clickjacking
helmet.hsts()	Enforce HTTPS
helmet.hidePoweredBy()	Hides "X-Powered-By: Express"

4.3 Customizing Helmet

```
app.use(helmet({
  contentSecurityPolicy: {
    useDefaults: true,
    directives: {
```

```
        "script-src": ["'self'", "https://cdn.example.com"]
      },
    },
    frameguard: {
      action: "deny"
    }
  }
});
```

4.4 Disable Specific Helmet Headers

```
app.use(helmet({
  frameguard: false,      // Disable frameguard
  contentSecurityPolicy: false
}));
```

5. Combining Rate Limiting + Helmet in a Production Express Server

```
const express = require("express");
const helmet = require("helmet");
const rateLimit = require("express-rate-limit");

const app = express();

// Helmet security
app.use(helmet());

// Global rate limiting
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 200
});
app.use(limiter);

// API routes
app.get("/api/data", (req, res) => {
  res.json({ message: "Secure API response" });
});

app.listen(3000, () => {
  console.log("Production server running");
});
```

6. Best Practices for Rate Limiting & Helmet

a) Best Practices for Rate Limiting

- Use Redis or Memcached for distributed deployments
 - Apply aggressive rate limiting on login and authentication routes
 - Log blocked requests for security audits
 - Use separate IP-based and user-based limits
 - Allow burst traffic but control sustained traffic
-

b) Best Practices for Helmet

- Enable CSP with strict rules
 - Enable HSTS for HTTPS-only apps
 - Restrict iframe usage
 - Block MIME type sniffing
 - Remove server identity headers
-